GOOGLE ADK CONTROL AUDIT

Public research edition | 28 October 2026

A bounded examination of tool confirmation and callback ordering

Finding in brief: In a local synthetic ADK test, a configured application callback changed the arguments of a resumed tool call after a confirmation request. A second conditional-confirmation test recorded an action after a rejection when the callback changed the argument used by the condition.

This research concerns a pinned open-source ADK snapshot and intentionally configured callbacks. It does not establish an external exploit, a production incident, or a failure of Google’s managed gateway.

1. What was examined

Grounded DI LLC examined the boundary between a confirmation request and a tool's eventual execution. The test used a mock model, an in-memory ADK runner, a synthetic tool, and application callbacks deliberately configured by the researchers. The synthetic tool recorded its effects in a Python list; no money moved and no external service was contacted.

The tested code was Google ADK Python commit **4c66a009106cbb855a79f4dc7383712057212e1e** (28 October 2026). This is a particular source snapshot, not a statement about every version or deployment.

2. Results

Local scenario	Observed result
Approve an unchanged call	The requested synthetic action was recorded.
Reject an unchanged call	No synthetic action was recorded.
Change an argument in a configured callback after approval	The synthetic action used the changed argument.
Change a conditional-confirmation argument after rejection	The synthetic action was recorded after the condition became false.
Change the confirmation record in a processor-level control	The mismatch raised an error; no action was recorded.

Two focused upstream test files passed 29 tests in this local environment. They do not establish that every possible callback configuration has been checked.

3. What this means—and what it does not

The source checks the recorded original call against the confirmation record. The relevant execution path subsequently allows before-tool callbacks to alter the arguments passed onward. In the conditional case, the confirmation predicate is evaluated on the later arguments. These are observations about the tested path and configuration.

A callback is application code with substantial authority. The tests deliberately installed one; they did not show that an outside party could install, control, or compromise one. The observations should not be read as a claim that Google's managed products, all ADK applications, or all confirmation configurations are affected.

4. Control considerations

For applications that require the confirmed action to remain fixed, one design option is to compare the final tool identity and effective arguments against the approved action immediately before execution. An explicit rejection can also be treated as controlling the pending request, even if a later condition changes. These are design considerations, not tested fixes or a statement of Google’s intended contract.

The boundary should also identify what the approval does and does not cover: application callbacks, tool implementation, identity, external policy state and downstream effects. Confirmation of a request alone does not prove that a remote action occurred exactly once.

5. Sources and scope

Pinned source: https://github.com/google/adk-python/tree/4c66a009106cbb855a79f4dc7383712057212e1e. Relevant files are the confirmation processor, tool caller, confirmation helper, and the repository’s ToolConfirmation guide. Google’s managed Agent Gateway documentation describes a separate gateway and is background only; no managed deployment was tested.

Tests ran locally on Python 3.12.14. The researchers used a Grounded DI local synthetic harness separately as audit infrastructure; it was not interposed in the tested ADK execution path. The findings do not demonstrate that the harness blocks either observation.

Unresolved: applicability to other commits and configurations, deployed user interfaces, remote tools, authenticated approval, gateway routing, concurrency and real external effects. No independent validation, vendor response or severity assessment is represented.

The accompanying publication verification record and checksums describe the release artifacts. Hashes identify bytes; they do not establish correctness or authorship.